

Using NeuroEvolution of Augmenting Topologies for a Free Fall Game

Ognjen Baucal
Computer science, Faculty of Science
University of Novi Sad

1 Problem description

1.1 Gameplay

The game consists of a player and clouds, the goal of the game is to not hit the clouds while falling down. The player can accelerate towards right and left to change its position. The player's y coordinate doesn't change, the clouds come up to him and as the game progresses and the score gets higher the clouds get faster and faster. The player is bound by the edges of the screen but isn't eliminated if he runs into them. The game runs at 30 frames per second.

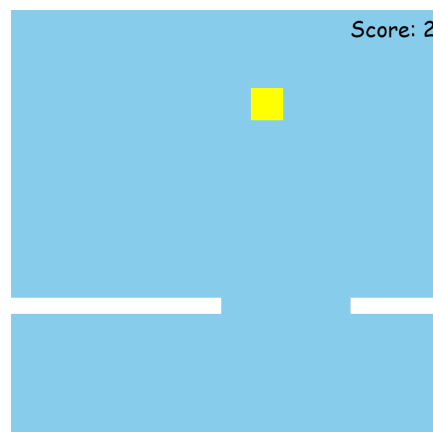


Figure 1: A snapshot of the in-game screen

1.2 State space

The first input is the player's x coordinate, ranging from $[0, 800]$. The second input is the player's velocity, since it's not moving up and down the velocity represents its horizontal velocity, it ranges between $[-50, 50]$, where the velocity represents the number of pixels traversed per frame. The third input is the distance between the player and the left side of the gap in the cloud, while the fourth input is the distance between the player and the right side of the gap in the cloud. Both distances can be negative to represent which side of the opening is the player on.

1.3 Action space

The action space is continuous, the agent outputs a number from $[-1, 1]$. The output is used as the acceleration that is then applied on the player's horizontal velocity.

2 Solution

2.1 NeuroEvolution of Augmenting Topologies

NeuroEvolution of Augmenting Topologies (NEAT) is a genetic algorithm that evolves neural networks. The algorithm changes the structure and the weights of neural networks, starting with a simple neural network with just input and output layers. Then adding on new connections, layers and changes to the weights and biases to discover new structures that might be good at solving a given task.

2.2 Evolution process

The NEAT algorithm begins with a population of simple neural networks, each neural network is encoded as a genome. The initial neural network has four inputs and one output, every neural network is set to use the Tanh activation function. The genome represents the network's structure, weights and biases. Then the networks are tested in their environment and each network is evaluated based on how well it performs, that measure is called fitness.

Networks with the highest fitness are selected to pass on their genes to the next generation. The algorithm uses a crossover process where genetic material is exchanged between two parent networks to create offspring. Mutations then alter the network's structure and weights. For example, new nodes and connections can be added, removed or modified. To preserve diversity in the population networks are grouped into species. This ensures that new and innovative structures are not discarded prematurely. Each species competes within its own group, reducing the risk of potential solutions being lost. The algorithm continues to evolve the population of neural networks through generations until it reaches a solution that meets the desired performance.

2.4 Fitness function

The fitness function evaluates the performance of a neural network. For every frame the player was alive it receives a reward of +0.1, this makes sure that players who survive longer have a higher fitness. When a player passes a cloud it receives a reward of +5 and when it hits a cloud it receives a punishment of -1.

3 Experiment

3.1 Experiment description

The fitness threshold was set at 900 which represents the player getting a score of 100. The NEAT algorithm returns the neural network which has the highest fitness evaluation. In order to find the best neural network for playing the game, the algorithm ran ten times with a population size at 50, and the maximum number of generations at 100. The best neural networks from all 10 runs were saved. To make sure their fitness evaluation wasn't influenced by luck, the networks were tested on 1000 more games and their average score was recorded to figure out which networks are truly the best while eliminating luck as a factor.

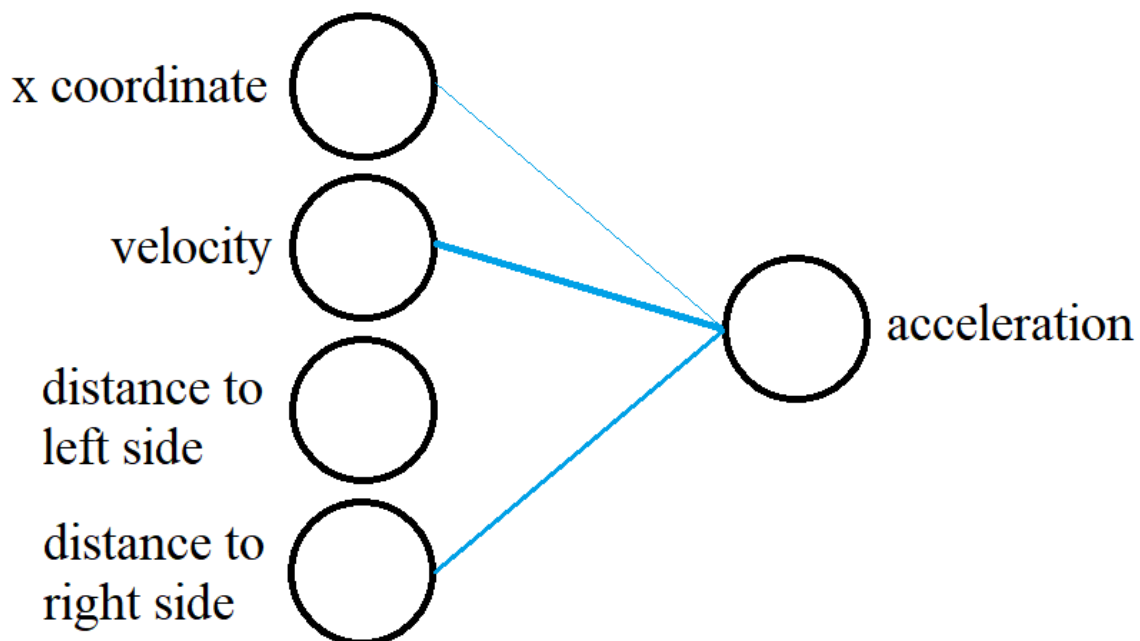
3.2 Experiment results

The best network, which had an average score that was five times higher than the second best ranking network, had an average score of 5096.2 over 1000 games. The network has no additional nodes and it has three connections, where the third input node that represents the distance between the player and the left side of the gap isn't connected to the output node.

Network parameters:

- bias of the output node is 0.43147542866685407
- weight of the first input is -0.06721720849042842
- weight of the second input is -1.7728214601625316
- weight of the fourth input is -0.22001059630553427

3.3 Visualisation



4 Conclusion

The NEAT algorithm can very quickly find an optimal solution for the given problem without having to tune hyperparameters or choose architectures. For this specific problem, the game isn't complicated enough to require a complex neural network structure.

Even though there were networks with additional nodes and connections, they were not as good at the game, as the best one was, by far. Furthermore, out of the ten chosen networks, seven of them had no additional nodes, just weighted connections from the input to the output layer.

References

[1] Kenneth O. Stanley and Risto Miikkulainen. "Efficient Evolution of Neural Network Topologies". 2009.